

RaftKV – Design Document

A from-scratch C++17 implementation of the Raft consensus algorithm backing a replicated key-value store. Five nodes agree on a single totally-ordered log and tolerate any minority failure (up to two of five) without losing an acknowledged write. No Raft library and no RPC/serialization framework are used – every RPC is a hand-rolled binary message over a plain TCP socket.

> Convert to PDF for submission with e.g. ``pandoc design.md -o design.pdf``.

1. Server architecture

Each node is one ``RaftServer`` process (``src/server.cpp``). All shared Raft state – ``role``, ``metadata`` (term/votedFor/commitIndex/lastApplied), the in-memory ``log_entries``, the per-peer ``nextIndex``/``matchIndex``, and the election timer – is protected by a single ``std::mutex raft_mutex``. The mutex is held only briefly around each state read/update and is always released across blocking network I/O.

Threads:

Thread	Count	Responsibility
client acceptor	1	<code>`accept()`</code> on the client port; spawns a handler thread per connection
client handler	1 / connection	parse text commands, drive PUT/GET/DELETE, reply
RPC receiver	1	<code>`accept()`</code> on the peer port; spawns a short-lived thread per inbound RPC
election timer	1	every 20 ms, if not leader and the deadline passed → start an election
per-peer replicator	1 / peer (4)	send AppendEntries (heartbeat or entries) to one peer, handle the reply

Two coordination condition variables:

- ``replicate_cv`` – kicks the per-peer replicators when there is new work (a fresh log entry, or a new leader). They also wake on a 100 ms timer to emit heartbeats.
- ``commit_cv`` – wakes client handler threads that are blocked waiting for their proposed entry to commit.

Handling each client connection in its own thread matters: a ``PUT`` blocks until its entry commits, and per-connection threads keep one slow/blocked client from stalling the others or the accept loop.

Two TCP ports per node: a **client port** speaking a line-based text protocol, and a **peer port** speaking the binary RPC protocol. They are kept separate so client traffic and Raft traffic never interleave on one socket.

2. RPC layer (``src/rcp.cpp``)

Every RPC is ``[5-byte header][body]``. The header is ``uint8`` type + ``uint32`` length (big-endian). ``sendRpc``/``recvRpc`` wrap ``sendAll``/``recvAll``, which loop until the exact byte count is transferred, so short reads/writes never corrupt a message.

Integers are encoded big-endian via ``pushU8/32/64`` / ``pullU8/32/64``. Bodies:

- `**RequestVoteArgs**` (32 B): ``term, candidateId, lastLogIndex, lastLogTerm``.
- `**RequestVoteReply**` (9 B): ``term, voteGranted``.
- `**AppendEntriesArgs**` (44 B + entries): ``term, leaderId, prevLogIndex, prevLogTerm, leaderCommit``, then a ``uint32`` entry count, then each entry as a length-prefixed blob produced by ``SerializeLogEntry``.
- `**AppendEntriesReply**` (9 B): ``term, success``.

Each connection carries exactly one request and one reply, then closes (one-shot). This is simple and, on localhost/LAN, fast enough; the cost is a TCP handshake per RPC, which only matters during long backtracking and is acceptable for the base project.

3. Persistent state & file format (``src/storage.cpp``, ``src/log.cpp``)

Two files per data directory:

`**`metadata.bin`**` – a fixed-size struct ``{ currentTerm, votedFor, commitIndex, lastApplied }`` (4 × ``uint64``). ``updateMeta`` rewrites the whole file and ``fsync``s it.

`**`logs.bin`**` – an append-only sequence of serialized log entries. Each entry (``SerializeLogEntry``):

...

```
term      uint64 (8B, big-endian)
index     uint64 (8B)
command   op(1B) | key_len(2B) | val_len(2B) | key bytes | value bytes
crc32     uint32 (4B)  # CRC32 over term+index+command
...`
```

The CRC lets recovery detect a torn write: ``DeserializeLogEntry`` validates each entry's CRC and stops at the first mismatch, returning the count of valid bytes. On startup the server truncates the file to that boundary, discarding a partial trailing entry from a crash mid-append.

Two write paths:

- ``writelog(entry)`` – append one entry + ``fsync``. Used by the leader for its own new entries (O(1)).
- ``rewriteLog(entries)`` – serialize the full log to ``logs.bin.tmp``, ``fsync``, then atomically ``rename`` over ``logs.bin`` (and ``fsync`` the directory). Used by a follower that must `**delete**` conflicting entries, which an append-only file cannot express. The temp-file + atomic-rename pattern means a crash during the rewrite never leaves a half-written log.

What is persisted, and when

``currentTerm``, ``votedFor``, and new log entries are written `**and `fsync`'d before`

the RPC that changed them replies** – the non-negotiable durability rule. If a node forgot its ``votedFor``/``currentTerm`` after a crash it could vote twice in one term and elect two leaders. ``commitIndex``/``lastApplied`` are **volatile** in Raft; we do persist ``commitIndex`` opportunistically (so restarts can replay the right prefix), but `**`lastApplied` is reset to 0 on boot**` and the state machine is rebuilt by replaying the log up to ``commitIndex`` – because the in-memory state machine is empty after a restart, trusting a persisted ``lastApplied`` would skip the replay and silently drop committed keys.

4. State machine (`src/statemachine.cpp`)

A `std::unordered_map<string,string>`. `apply(cmd)` does PUT (insert/overwrite), DELETE (erase), or NOOP (nothing). It is fully deterministic: the same committed log replayed on any node yields identical state – the property that turns the replicated log into a replicated database. `dump` (sorted key=value) and `hash` (count + CRC32 over the sorted set) expose the local applied state for testing.

5. RequestVote receiver (§7.1, `handleRequestVote`)

1. If `args.term < currentTerm` → reply `{currentTerm, false}`.
2. If `args.term > currentTerm` → `stepDown(args.term)`: become follower, set term, clear `votedFor`, `**persist**`.
3. Grant iff (`votedFor` is 0 or already this candidate) `**and**` the candidate's log is at least as up-to-date (election restriction §5.4.1: higher `lastLogTerm` wins; on a tie, the longer log wins). On grant: set `votedFor`, `**persist (fsync) before replying**`, and reset the election timer.
4. Otherwise reply `false`.

The persist-before-reply in step 3 is what makes "one vote per term" survive a crash.

6. AppendEntries receiver (§7.2, `handleAppendEntries`) – all six steps

1. If `args.term < currentTerm` → reply `false`.
2. `args.term >= currentTerm` → recognize the leader: bump+persist `currentTerm` if higher, become FOLLOWER, record `leaderId` (for client redirects), reset the election timer.
3. `**Log matching**`: if `prevLogIndex > 0` and either our log is shorter than `prevLogIndex` or `term@prevLogIndex != prevLogTerm` → reply `false`.
4. `**Conflict**`: for each incoming entry, if an entry already exists at that index with a different term, truncate from that index and append the leader's entry.
5. `**Append**` any genuinely new entries.
6. If `leaderCommit > commitIndex`, set `commitIndex = min(leaderCommit, index of last new entry)`, then apply newly-committed entries.
7. Reply `true`.

If steps 4/5 changed the log, the whole log is re-persisted with `rewriteLog` (needed because step 4 deletes entries). The hard part isn't any single step – it's never skipping the persist or the timer reset on any branch.

7. Commitment rule (§9.3, `advanceCommit`) and the current-term restriction

After any `matchIndex` update the leader scans from its last index downward for the highest `N > commitIndex` such that (a) a majority (itself + followers with `matchIndex >= N`) stores `N`, `**and**` (b) `term@N == currentTerm`. It then sets `commitIndex = N` and applies up to it.

Condition (b) is essential. Figure 8 of the Raft paper shows that a leader which commits an entry from a *previous* term purely because it is now majority-stored can later have that entry overwritten – a committed entry lost, violating

safety.

A leader may only directly commit entries from its own term; older entries commit

transitively once a current-term entry above them commits (log matching makes everything below it identical across the majority). The unit test ``test_commitment_rule`` encodes exactly this Figure-8 case.

8. The NOOP trick (§7.4, ``becomeLeader``)

A freshly elected leader does not know which entries from prior terms are committed, and rule (b) above forbids it from committing them directly. So immediately on winning, the leader appends a ****NOOP**** entry in its new term and replicates it. Committing that NOOP (it *is* in the current term) transitively commits every preceding entry – unblocking any client that was waiting on an entry stranded by the previous leader's crash. Without this, certain failure patterns leave clients hanging forever even though safety is intact.

9. Catch-up backtracking (``replicationLoop``)

On election the leader sets ``nextIndex[peer] = lastLogIndex + 1`` and ``matchIndex[peer] = 0`` for every peer. Each per-peer replicator builds an `AppendEntries` from ``nextIndex[peer]``:

- ****success**** → ``matchIndex = prevLogIndex + #entries``, ``nextIndex = matchIndex + 1``, then run the commitment scan. If caught up, sleep until the next kick/tick.
- ****failure, same-or-lower term**** (log mismatch) → decrement ``nextIndex`` and retry immediately (the fast inner loop, no 100 ms wait), walking backward one entry at a time until a matching ``prevLogIndex/prevLogTerm`` is found; the next success then ships all remaining entries in one message.
- ****failure, higher term**** → step down to follower.

This is the simple Figure-2 backtracking (one entry per probe). The optimized term-skipping variant is a listed bonus and intentionally not implemented.

A leader proposing a client command appends locally, ``fsync``s, kicks ``replicate_cv``, and blocks the client thread on ``commit_cv`` until ``commitIndex`` reaches its entry (or it loses leadership / times out at 3 s, in which case the client gets ``NOT_LEADER`` or a timeout error and retries).

10. Client interaction (§9.4)

Clients connect to any node. A non-leader answers ``PUT/GET/DELETE`` with ``NOT_LEADER leader=N`` (the id it last heard in an `AppendEntries`) or ``UNKNOWN_LEADER`` during an election; the redirect-aware ``raftkv-cli`` reconnects to ``N`` (or rotates) and retries. ****Reads go through the log****: a ``GET`` appends a NOOP, waits for it to commit, then returns the value – linearizable, at the cost of a log round-trip (the lease-based fast read is intentionally not used). Client responses are written with a ``sendAll`` loop so large replies (``\dump``) are never truncated by a short write.

11. Chaos test walkthrough (``chaos/chaos_test.sh``)

Pure bash, no Python. After each step it compares every alive node's ``\hash``

(server-side count + CRC of applied state) and asserts agreement.

1. ****Steady state**** – start 5, elect a leader, submit 1000 PUTs through the redirecting client. Expect 1000/1000 OK and all five nodes identical.
2. ****Leader failover**** – kill the leader, wait for a new one, submit 1000 more PUTs, restart the killed node. It catches up via backtracking; all five end identical at 2000 keys. (This is where resetting `lastApplied` on recovery is load-bearing – see §3.)
3. ****Follower loss with majority**** – kill two followers, submit 500 PUTs (3/5 is still a majority), restart them; they catch up; all agree.
4. ****Loss of majority**** – kill three nodes (only two alive). 100 PUTs must all fail/time out because no majority is possible (the safety behavior students most often get wrong). Restart one node → majority restored → 100 PUTs commit.
5. ****Partition**** – isolate two non-leader nodes; the majority side keeps accepting writes, the minority side accepts none; heal and verify catch-up. (Isolation is kill-based; a true bidirectional partition needs `iptables`/root – noted as a limitation.)
6. ****Crash mid-batch**** – kill the leader partway through 1000 PUTs. After recovery the committed count is within a small margin of 1000 with ****no gaps**** in the committed prefix, and all nodes agree.

12. Safety properties and where they live

- ****Election safety (≤ 1 leader/term)**** – majority vote + one persisted vote per term: `handleRequestVote` (persist `votedFor` before replying) and the majority check in `sendRequestVoteToAll` → `becomeLeader`.
- ****Leader completeness**** – the election restriction in `handleRequestVote` (up-to-date log check) guarantees a new leader holds every committed entry.
- ****Log matching**** – the `prevLogIndex/prevLogTerm` consistency check plus conflict-delete in `handleAppendEntries` (steps 3–5).

13. Limitations

No snapshot/log compaction, no pre-vote, simple (non-term-skipping) backtracking, log-based (non-lease) reads, fixed cluster size of five. All are explicitly out of base scope; the first three are listed bonuses.